Assessment tool-01

Name: D.Suresh

Reg.No: 192525305

course Name : Data Structures

course code : CSA0307

1. Convert the following infix expression into postfix notation using a stack. show the contents of the stack after processing each symbol.

(A+B) ×(C-D)/E

* Convert the infix expression into postfix using a stack

Infix expression:

(A+B) ×(C-D)/E

step-by-step stack contents

Symbol	stack	postfix
(	(	
A	(	A
+	(+	A
B	(+	AB
)	Empty	AB+
×	×	AB+
(	×(	AB+
C	×(	AB+C

	×(-	AB+C
-	×(-	AB+CD
D	×	AB+CD-
)	/	AB+CD-×
/	/	AB+CD×E
E		AB+CD-×E/
End	Empty	

Final Postfix Expression:

AB+CD-×E/

2. Convert the following infix expression into postfix form. Explain how operator precedence and associativity influence the conversion

A+B*C-(D/E+F)*G

* convert the following infix expression into postfix

Infix expression:

A+B*C-(D/E+F)*G

Conversion

1. B×C → BC*
2. A+(BC)* → ABC*+
3. D/E → DE/

4. (D/E) + F → DE/F+

5. (DE/F+) * G → DE/F+G*

6. ABC*+ − DE/F+G** → ABC*+DE/F+G*−

Final Postfix Expression

ABC*+DE/F+G*−

Operator Precedence and Associativity:-

* Parentheses ( ) have the highest priority
* Multiplication (*) and division (/) have higher precedence than Addition (+) and subtraction (−)

* Associativity:-

  - +, −, *, / are left-to-right associative
  - Operators of the same precedence are evaluated from left to right.

⇒ These rules ensure that multiplication and division are converted before addition and subtraction, while parentheses are processed first